



UNIVERSIDAD NACIONAL DEL LITORAL
Facultad de Ingeniería y Ciencias Hídricas

ANTEPROYECTO

**Análisis e Implementación de
resolvedores lineales en GPGPU
aplicados a OpenFOAM**

Director

Norberto NIGRO

Ayudantes

Autor

Juan Pablo A. LESCANO

Mag. Ing. Juan Marcelo
GIMENEZ

Dr. Ing. Santiago MARQUEZ
DAMIAN

Resumen

Un análisis comparativo de dos implementaciones del método del gradiente conjugado preconditionado (PCG) sobre unidades de procesamiento gráfico de propósito general (GPGPU) utilizando Open Computing Language (OpenCL) aplicados a OpenFOAM[®] es propuesto para determinar la mejor alternativa en términos de tiempo y consumo energético. Por un lado, un desarrollo propio utilizando las mejoras en el manejo de memoria introducidas por OpenCL 2.0; por otro, la solución provista por Paralution[®], la cual utiliza una versión anterior de OpenCL. Ambas se compararán entre si, y frente a la solución paralela nativa de OpenFOAM[®] mediante la simulación de la aerodinámica de un vehículo de competición.

Para ello, se establece una línea base, a partir de una metodología propia y su descomposición en tareas, comenzando con la implementación del PCG de Paralution[®]. Luego, el estudio, y análisis de, por un lado, programación paralela en OpenCL 2.0 y, por otro, el PCG y su posterior diseño y desarrollo. Finalmente, las tareas de cierre, recopilación de datos y escritura del informe final. La primera etapa presentará 2 entregables, en momentos distintos; mientras que las dos últimas presentarán un informe de avance cada una.

El análisis y planificación de cada tarea definen una línea base del cronograma, marcando el camino crítico. El análisis de costo tuvo en cuenta los servicios, bienes de capital, necesarios y disponibles, con su correspondiente amortización, mostrando solo los resultados de la misma.

Finalmente, la identificación y análisis, tanto cualitativo como cuantitativo, de los riesgos definen estrategias de contingencia, mitigación o aceptación.

Palabras clave— GPGPU, CFD, OpenFOAM, Paralution, OpenCL, SVM, OpenSource, Libre

Índice

1. Justificación	3
2. Objetivos	4
3. Alcance	4
3.1. Funcionales	4
3.2. No funcionales	5
3.2.1. Instalación y puesta a punto del Sistema Operativo Linux, en su distribución Ubuntu 14.04	5
3.2.2. Uso óptimo de los recursos de hardware disponibles para obtener mejoras en	5
3.2.3. Uso de buenas prácticas de programación:	5
3.3. Exclusiones	5
3.4. Limitaciones y Restricciones	5
3.5. Supuestos	6
3.6. Criterios de aceptación del producto	6
4. Metodología	6
4.1. Análisis y desarrollo de los métodos	6
4.2. Implementación y prueba de los métodos	6
4.3. Escritura del informe final y cierre	6
5. Plan de Tareas	6
5.1. Análisis y desarrollo de los métodos	7
5.1.1. Análisis de los solvers de Paralution®	7
5.1.2. Puesta en funcionamiento de los solvers de Paralution®	7
5.1.3. Estudio del marco teórico de arquitecturas paralelas y OpenCL 2.0	7
5.1.4. Análisis del método PCG	7
5.1.5. Desarrollo del método PCG en OpenCL 2.0	7
5.1.6. Finalizar y Documentar la fase	7
5.2. Implementación y prueba de los métodos	7
5.2.1. Implementación	7
5.2.2. Prueba	7
5.2.3. Finalizar y Documentar la fase	8
5.3. Escritura del informe final y cierre	8
5.3.1. Documentación	8
5.3.2. Entrega del producto	8
5.4. Consideraciones	8
6. Cronograma	8
7. Entregables	10
8. Recursos	11
8.1. Disponibles	11
8.1.1. Bienes de Capital	11
8.1.2. Recursos Humanos	11
8.2. Necesarios	11
8.2.1. De Capital	11
8.2.2. Servicios	11
8.2.3. Insumos	11
8.2.4. Otros	11

9. Presupuesto	11
10. Riesgos	12

1. Justificación

En la antigüedad, la investigación de la dinámica de fluidos se realizaba solamente a través de la experimentación y observación del flujo de fluidos en entornos físicos. El desarrollo de la computación dió lugar a la creación de la dinámica de fluidos computacional (CFD), es decir, la simulación de fluidos con modelos y métodos numéricos en entornos virtuales.(Kimbrell, 2012).

Open Source Field Operation And Manipulation (OpenFOAM[®]) es un paquete de software CFD modular basado en el método de volúmenes finitos (FVM) como formulación a parámetros distribuidos; con una base de usuarios en constante crecimiento, a través de múltiples áreas de la ingeniería y la ciencia. Dicho paquete cuenta con una variedad de aplicaciones para la resolución de los problemas típicos de la CFD, incluidas herramientas para la generación de mallas para la representación de volúmenes; resolvedores de ecuaciones diferenciales en derivadas parciales (PDE); resolvedores de sistemas lineales de ecuaciones (SLE); utilidades de visualización, pre y post-procesamiento, entre otras (Greenshields, 2015).

El Centro de Investigación de Métodos Computacionales (CIMEC) se dedica a la implementación y el uso de métodos computacionales en la ciencia e ingeniería utilizando, entre otras herramientas de software, OpenFOAM[®] para llevar a cabo las simulaciones de, por ejemplo, motores a explosión, turbinas eólicas, problemas de la industria aeronáutica, petrolera, siderúrgica, nuclear, etc. (CIMEC, 2015)

La observación del proceso de simulación indica que, aproximadamente, el 80 % del tiempo se emplea en la resolución de los SLE, siendo el mismo distribuido y ejecutado por las unidades centrales de procesamiento (CPU). Sin embargo, el volumen de datos a procesar suele ser tan grande, que puede demorar desde días hasta semanas en completarse. Por otra parte, el aumento exponencial en la capacidad de cómputo de las unidades de procesamiento gráfico de propósito general (GPGPU) frente a las CPUs (NVIDIA Corporation, 2015), siendo los SLE una de las pocas etapas de la simulación de índole repetitiva, con nula casuística, resulta muy atractiva para ser resuelta con estas unidades de procesamiento. Por ejemplo, una AMD R9 Fury X posee una capacidad de cómputo de 8.6 TeraFlops (Advanced Micro Devices, Inc, 2015), frente a los 182 GigaFlops del Intel Core i7 4770K o los 354 GigaFlops de un Core i7 5960X. (Intel Corporation, 2015).

Open Computing Language (OpenCL) es un estándar, conformado por una interfaz de programación de aplicaciones (API) y un lenguaje de programación, que permite hacer uso de las unidades de GPGPU. En su versión 2.0 incorpora múltiples mejoras respecto a las versiones anteriores, que simplifican el manejo de los espacios de memoria a su vez que minimizan las transferencias entre los dispositivos a través de un esquema de memoria virtual compartida (SVM) (Kaeli et al., 2015).

Actualmente, se encuentran resolvedores de SLE implementados en GPGPU con OpenCL, sin embargo, no todas son libres y de código abierto; y su número disminuye para los compatibles con OpenFOAM[®]; Entre ellas, destaca la implementación de Paralution[®], tanto por ser de código abierto, independiente de la plataforma y el sistema operativo, como por presentar un soporte multi-GPU maduro, estable y testeado(PARALUTION Labs UG, 2015). Al momento todas estas implementaciones no han dado el speed-up esperado en las aplicaciones que se consideran de interés para este proyecto. En gran medida por el manejo de la limitada memoria con que cuentan.(Lukarski, 2013) Finalmente, no existen en el mercado soluciones que hagan uso de OpenCL 2.0, estando Paralution[®] implementado en OpenCL 1.2.

Por otra parte, de la experiencia de los cálculos realizados en el CIMEC surge que la resolución de problemas que involucran la presión, como el caso de los flujos incompresibles, son los que consumen sustancialmente mas tiempo en comparación a otras simulaciones, usando el método de gradiente conjugado precondicionado (PCG), principalmente, para resolver estos SLEs.

Es por todo lo anterior, que se propone el diseño y desarrollo del método PCG en GPGPU utilizando OpenCL 2.0, y la comparación con Paralution[®], esperando obtener mejoras en el tiempo de ejecución al implementarlo sobre un caso acerca de la aerodinámica de un vehículo de competición.

Finalmente, el CIMEC dispondrá de una nueva alternativa para realizar las simulaciones en un tiempo que se espera sea sensiblemente inferior. Además, se pretende que el producto final sea Open Source, de acceso, utilización y distribución libre y gratuito, llegando de esta manera a la comunidad de usuarios de OpenFOAM[®] y todo lo que esto trae aparejado. En lo personal, se espera adquirir conocimientos tanto en el uso de OpenFOAM[®] como en la programación paralela en general, y OpenCL 2.0 en particular, el cual se perfila a ser el estándar por defecto.

2. Objetivos

A continuación se listan los objetivos generales en conjunto con sus objetivos específicos:

- Diseñar y desarrollar el método de PCG en GPGPU con OpenCL 2.0 para OpenFOAM[®].
 - Analizar la comunicación con OpenFOAM[®].
 - Analizar el paradigma de programación paralela.
 - Analizar la API OpenCL 2.0
 - Analizar el método PCG.
 - Validar el método PCG.
- Analizar e implementar los solvers lineales para OpenFOAM[®] de Paralution[®].
 - Analizar el uso de la librería Paralution[®]
 - Implementar el resolutor PCG de Paralution[®].
- Desarrollar una aplicación para resolver un problema acerca de la aerodinámica de vehículos de competición.
 - Obtener los datos del modelo de aerodinámica
 - Obtener diferentes soluciones con los distintos solvers desarrollados.
 - Seleccionar la mejor alternativa en cuanto a eficiencia flops/watts y tiempo/watts.

3. Alcance

El alcance del proyecto se encuentra especificado según los requerimientos funcionales y los no funcionales, al mismo tiempo que se establecen los límites y exclusiones, como se detalla a continuación.

3.1. Funcionales

- Implementación del solver PCG de Paralution[®] en OpenCL para ser utilizados en OpenFOAM[®].
- Implementación propia del PCG sobre OpenCL 2.0 para matrices LDU compatible con el formato de OpenFOAM[®].
- Evaluación de convergencia de los métodos a la aproximación de solución de la ecuación diferencial.
- Estudiar la eficiencia y efectividad de todos los métodos.

3.2. No funcionales

3.2.1. Instalación y puesta a punto del Sistema Operativo Linux, en su distribución Ubuntu 14.04

- Instalación del S.O.
- Instalación de los drivers de la GPU AMD R9 290.
- Compilación e instalación de:
 - OpenFOAM[®]
 - Paralution[®]
 - AMD APP SDK 3.0

3.2.2. Uso óptimo de los recursos de hardware disponibles para obtener mejoras en

- Rendimiento
- Escalabilidad
- Estabilidad
- Confiabilidad

3.2.3. Uso de buenas prácticas de programación:

- Uso de reglas de escritura de código específicas para OpenFOAM
- Código legible y mantenible.
- Uso de un repositorio de GitHub para versionado de código.
- Tests unitarios sobre cada método principal.

3.3. Exclusiones

- Sólo se hará uso de software libre.
- No se harán comparativas sobre solvers propietarios.
- Sólo se implementarán solvers OpenSource que trabajen sobre OpenCL.

3.4. Limitaciones y Restricciones

- Se dispone de una única GPU AMD Radeon R9 290 con 4GB de RAM GDDR5.
- Se trabajará con problemas de tamaño máximo abordable por el hardware disponible.
- Se trabajarán todos los datos en memoria RAM, tanto del host como de los dispositivos.

3.5. Supuestos

- Se dispone de todos los recursos de hardware y software necesario para desarrollar sobre OpenCL 2.0.

3.6. Criterios de aceptación del producto

Mínimo: lograr una mejora de eficiencia y performance respecto a las soluciones provistas nativamente por OpenFOAM®.

Esperado: Lograr una mejora de performance respecto a las soluciones provistas por Paralution®.

4. Metodología

Dada la limitación del personal disponible encargada de las tareas del proyecto, se planifica su realización de forma secuencial con relación terminar para empezar (TE) a través de una metodología propia definida para el proyecto. El proyecto se dividirá en tres etapas:

4.1. Análisis y desarrollo de los métodos

En esta primera fase se realizará por un lado, las tareas de análisis de las soluciones disponibles en el mercado de Paralution® para OpenFOAM®, su linkeo, compilación, utilización, configuraciones y pruebas en casos simples para verificar su funcionamiento; por otro lado, se realizará un estudio sobre cálculo paralelo y el método del gradiente conjugado, su utilización y desarrollo en OpenCL 2.0 y pruebas sobre casos simples con el mismo propósito. Al finalizar cada una de estas subetapas, se realizará un informe y un prototipo de resolutor en condiciones de ser utilizado.

4.2. Implementación y prueba de los métodos

En esta segunda fase se realizarán las configuraciones de OpenFOAM® y puesta a punto, para correr la simulación sobre un caso real vehículos de competición, utilizando el solver propio y el de Paralution®. Se registrarán todos los resultados obtenidos en cuanto a tiempo transcurrido, consumo energético y se compararan entre si. Al finalizar se documentarán los resultados en un informe.

4.3. Escritura del informe final y cierre

En esta ultima fase, se realizarán todas las tareas referidas a la escritura del informe final y cierre del proyecto. Se revisarán todos los resultados obtenidos y se elaborarán las conclusiones determinando cual de los métodos es el mejor, teniendo como criterio el tiempo total de corrida de la simulación y su consumo energético.

5. Plan de Tareas

El plan de tareas se define en base a las etapas expuestas en la sección Metodología, P. 6. Las duraciones de las tareas se estimaron en base a criterios de experto por parte del director y los ayudantes.

A continuación se detallan los niveles del plan de tareas.

5.1. Análisis y desarrollo de los métodos

5.1.1. Análisis de los solvers de Paralution[®]

- Análisis de la utilización del resolutor PCG de Paralution[®]. (16 Horas)

5.1.2. Puesta en funcionamiento de los solvers de Paralution[®]

- Linkar y compilar el resolutor PCG de Paralution[®]. (8 Horas)
- Probar el solver de Paralution[®]. (8 Horas.)
- Documentar los resultados. (15 Horas.)
- Presentar los resultados. (Hito)

5.1.3. Estudio del marco teórico de arquitecturas paralelas y OpenCL 2.0

- Revisión de literatura sobre arquitecturas paralelas. (27 Horas)
- Revisión de literatura sobre OpenCL 2.0. (48 Horas)
- Realización de ejercicios prácticos sobre OpenCL 2.0. (8 Horas)

5.1.4. Análisis del método PCG

- Revisión de literatura acerca del método PCG. (12 Horas)
- Diseñar el método sobre OpenCL 2.0. (40 Horas)

5.1.5. Desarrollo del método PCG en OpenCL 2.0

- Desarrollar el método PCG sobre OpenCL 2.0. (48 Horas)
- Optimizar el código. (48 Horas)
- Realizar corridas de prueba y validar los resultados. (8 Horas)

5.1.6. Finalizar y Documentar la fase

- Recolectar, organizar y clasificar la información. (12 Horas)
- Documentar la información. (16 Horas)
- Presentar entregables (Hito)

5.2. Implementación y prueba de los métodos

5.2.1. Implementación

- Configurar el entorno donde se correrán las simulaciones. (4 Horas)
- Obtener los datos de la simulación. (1 Hora)
- Crear los archivos de configuración de OpenFOAM[®] para el resolutor de Paralution[®]. (4 Horas)
- Crear los archivos de configuración de OpenFOAM[®] para el resolutor propio. (4 Horas)

5.2.2. Prueba

- Correr la simulación con los resolutores de Paralution[®]. (12 Horas)
- Correr la simulación con el resolutor propio. (6 Horas)
- Verificar y comparar los resultados de las simulaciones. (11 Horas)

5.2.3. Finalizar y Documentar la fase

- Recolectar, organizar y clasificar los resultados. (11 Horas)
- Documentar los resultados. (11 Horas)
- Entregar la documentación y las configuraciones utilizadas. (Hito)

5.3. Escritura del informe final y cierre

5.3.1. Documentación

- Documentar el resolutor desarrollado. (15 Horas)
- Recolectar, organizar y clasificar la información. (12 Horas)
- Escribir el Informe Final. (40 Horas)

5.3.2. Entrega del producto

- Obtener la aceptación de los resolutores por parte del CIMEC. (2 Horas)
- Presentar el producto final. (3 Horas)
- Entregar el producto final y las conclusiones. (Hito)

5.4. Consideraciones

Se dedicará un esfuerzo temporal de 20 Horas semanales, divididas a lo largo de la semana, donde se espera trabajar de Lunes a Viernes. Por otra parte, cabe destacar que las pruebas unitarias se encuentran implícitas en la labor del desarrollo.

6. Cronograma

Para su elaboración se tuvieron en cuenta las limitaciones de los recursos humanos asignados al proyecto, requiriendo que las tareas se ejecuten de forma secuencial, y que por lo tanto, solo exista un único camino, coincidente con el camino crítico. Se propone el 3 de Agosto de 2015 como fecha de inicio del proyecto, con una carga de 4 horas diarias, de lunes a viernes. Solamente se considera el 25 de Diciembre como feriado no laborable. Se estima el siete de Enero de 2016 como fecha de finalización del mismo, con una duración total de 450 horas. En la figura 1 se presenta el diagrama de Gantt, donde se expresan las relaciones entre tareas y su duración estimada.

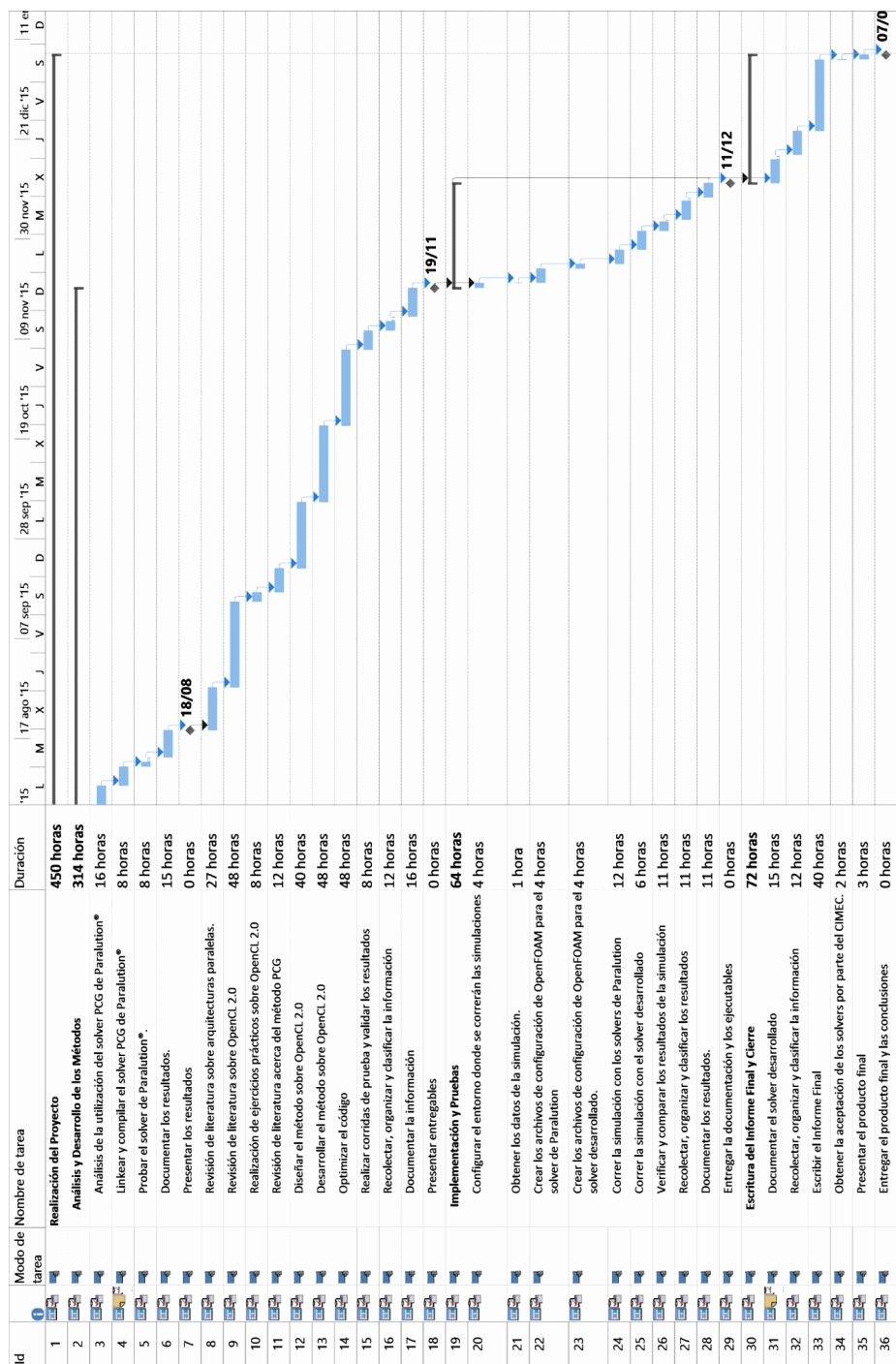


Figura 1: Diagrama de Gantt

7. Entregables

Durante la ejecución del proyecto, se planean realizar 4 entregables. El primero se corresponde con el prototipo utilizando el resolvidor PCG de Paralution[®] y su informe correspondiente. El segundo con el resolvidor PCG propio y el informe. El tercero con un informe de implementación de ambos resolvidores sobre un caso de simulación real. Finalmente, un cuarto informe recolectando toda la información obtenida hasta el momento y las conclusiones logradas. Para cada uno se detalla una fecha estimativa de entrega, su alcance y los criterios de aceptación.

1.
 - Fecha: 18/08/2015
 - Descripción: Informe y Primer Prototipo.
 - Contenido del Informe: Descripción de la utilización de los resolvidores de Paralution[®]; Descripción para la compilación de los resolvidores de Paralution[®]; Resultados preliminares de las pruebas;
 - Alcance del prototipo: linkear y compilar el resolvidores lineal PCG de Paralution[®] en Ubuntu 14.04; Correr casos de prueba simples.
 - Criterio de aceptación: El director será quien determine si el prototipo funciona correctamente.
2.
 - Fecha: 19/11/2015
 - Descripción: Informe y Segundo Prototipo.
 - Contenido del Informe: Documento con conceptos sobre arquitecturas paralelas y OpenCL 2.0; Resultados preliminares de la aplicación de OpenCL 2.0; Descripción del método PCG; Descripción de la interfaz con OpenFOAM[®]; Diseño y código fuente del resolvidor; Resultados de la Optimización; Comparativa sobre casos simples.
 - Alcance del prototipo: Linkear y compilar el resolvidor PCG desarrollado en Ubuntu 14.04; Correr casos de prueba simples.
 - Criterio de aceptación: El director será quien determine si el prototipo funciona correctamente.
3.
 - Fecha: 11/12/2015
 - Descripción: Informe y Producto Final.
 - Contenido: Documento con los resultados obtenidos de aplicar los métodos sobre la simulación de los automóviles de competición, tiempos requeridos, consumos de energía, comparativa entre los resultados; Ejecutables, Código fuente y archivos de configuración.
 - Criterio de aceptación: Lograr la misma solución con todos los resolvidores.
4.
 - Fecha: 01/02/2016
 - Descripción: Informe.
 - Contenido: Documento con la recopilación de todos los resultados previos. Observaciones, Conclusiones, trabajos futuros.
 - Criterio de aceptación: Deberá poseer todos los resultados previamente alcanzados, con sus respectivas conclusiones. Las observaciones y los trabajos futuros serán opcionales.

8. Recursos

8.1. Disponibles

8.1.1. Bienes de Capital

1. PC de Escritorio (Intel Core i7 870, 12GB RAM DDR3, AMD Radeon R9 290, Monitor 22", Mouse y Teclado)
2. Laptop (Intel Core i5 2540M, 4GB DDR3, HD3000, Monitor 15.6")
3. Sistema Operativo Ubuntu 14.04
4. Compiladores C++.
5. SDK de OpenCL 2.0
6. Pizarra
7. Fibrones
8. Impresora Monocromática con Sistema Continuo.

8.1.2. Recursos Humanos

1. Desarrollador
2. Director
3. Ayudantes

8.2. Necesarios

8.2.1. De Capital

1. Escritorio
2. Silla de Escritorio Acolchonada

8.2.2. Servicios

1. Servicio de Electricidad
2. Servicio de Agua
3. Servicio de Internet

8.2.3. Insumos

1. Resmas de hojas A4
2. Tinta negra para Fibrones
3. Tinta negra para Impresora
4. Productos de Limpieza

8.2.4. Otros

1. Fotocopias
2. Encuadernación

9. Presupuesto

El análisis de costos del proyecto se detalla a continuación:

Cuadro 1: Presupuesto

Descripción	Unidades	Precio Unitario [\$]	Costo Total [\$]
PC de escritorio	1	12000	12000
Laptop	1	6000	6000
Pizarra	1	250	250
Fibrones	2	20	40
Impresora	1	1200	1200
Desarrollador	450 Horas	100	45000
Director	100 Horas	150	15000
Ayudante 1	80 Horas	120	9600
Ayudante 2	80 Horas	120	9600
Escritorio	1	1200	1200
Silla de Escritorio	1	1200	1200
Servicio de Electricidad	6 Meses	260 bimestral	780
Servicio de Agua	6 Meses	450 bimestral	1350
Servicio de Internet	6 Meses	280 mensual	1680
Resma de Hojas A4	2	70	140
Tinta para Fibrones	2	40	80
Tinta para Impresora	100ml	60	60
Encuadernación	3	250	750
		Total	104850

10. Riesgos

Nombre: Estimación incorrecta de los tiempos en el plan de tareas.

Descripción: El riesgo puede ocurrir como una subestimación o una sobreestimación de los tiempos del plan de tareas.

Indicador: Error relativo porcentual entre el progreso estimado de cada tarea para cada día y el real.

Probabilidad de ocurrencia: 3

Impacto: Medio, el proyecto se puede retrasar o adelantar

Estrategia a adoptar: Si el indicador es mayor al 10 %, se trabajarán horas extras para recuperar el progreso perdido y respetar el calendario de tareas.

Si el indicador es mayor al 35 % se deberán replanificar las tareas que se encuentren en etapa de ejecución y las siguientes a la misma.

Se busca mitigar el impacto sobre las fechas de entrega para que la diferencia entre el momento planificado y el real sea mínima.

Nombre: Estimación incorrecta de la complejidad del solver a desarrollar.

Descripción: Se estima que los recursos disponibles, humanos y bibliográficos principalmente, sean suficientes para el diseño y desarrollo del solver.

Probabilidad de ocurrencia: 1.

Indicador: Solo observable en la etapa de diseño y desarrollo del solver. Se monitoreará paralelamente el progreso de la etapa.

Impacto: Medio, afecta las fechas de entrega de la etapa 2 y posteriores.

Estrategia: Aceptar activamente. Se consultará con expertos en la temática.

Nombre: Ausencia de los RRHH

Descripción: El riesgo ocurre ante la ausencia de parte o totalidad del personal del proyecto.

Probabilidad de ocurrencia: 2.

Indicador: Cantidad de personas disponibles, haciendo énfasis en el desarrollador.

Impacto: Medio. El proyecto puede detenerse hasta el regreso del desarrollador, pero puede continuar ante la ausencia del director o los ayudantes.

Estrategia: Ante la ausencia del desarrollador, se aceptará activamente, y se replanificarán las tareas. Ante la ausencia de director y/o ayudantes se aceptará pasivamente.

Nombre: Rotura de la GPU

Descripción: El riesgo ocurre ante la rotura de la GPU.

Probabilidad de ocurrencia: 1.

Indicador: Visual, si la placa deja de dar señal.

Impacto: Medio o Bajo. El proyecto se puede retrasar en todas las tareas que demanden el uso de la GPU. Es decir, las de desarrollo, pruebas e implementación. Fuera de esas categorías, se puede continuar trabajando.

Estrategia: En caso de rotura en las etapas antes mencionadas, se comprará una segunda unidad de forma inmediata. Se replanificarán las tareas acorde al tiempo de adquisición de la placa. En caso de rotura fuera de dichas etapas, también se adquirirá una segunda unidad, pero evaluando el tiempo disponible hasta la tarea que la requiera.

Referencias

- Advanced Micro Devices, Inc (2015). Amd radeon™ r9 series graphics cards.
- Burden, R. L. and Faires, J. D. (2002). *Análisis numérico 7a. ed.* Brooks/Cole.
- CIMEC (2015). Centro de investigación de métodos computacionales. [Online; accessed 25-July-2015].
- Ghysels, P. and Vanroose, W. (2014). Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224 – 238. 7th Workshop on Parallel Matrix Algorithms and Applications.
- Greenshields, C. J. (2015). *OpenFoam - User Manual*. OpenFOAM Foundation Ltd.
- Howes, L. and Munshi, A. (2014). *The OpenCL Specification Version 2.0*. Khronos Group.
- Intel Corporation (2015). Ark - su fuente de información de productos intel®.
- Kaeli, D., Mistry, P., Schaa, D., and Zhang, D. (2015). *Heterogeneous Computing with OpenCL 2.0*. Elsevier Science.
- Kimbrell, A. B. (2012). Development and verification of a navier-stokes solver with vorticity confinement using openfoam. Master’s thesis, University of Tennessee.
- Lukarski, D. (2013). Paralution - a library for iterative sparse methods on cpu and gpu.
- NVIDIA Corporation (2015). *CUDA C PROGRAMMING GUIDE*. NVIDIA Corporation.
- PARALUTION Labs UG (2015). *Paralution - User Manual*. PARALUTION Labs UG.
- Project Management Institute (2013). *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*. PMI Standard. Project Management Institute, Incorporated.
- Sommerville, I. (2011). *Software Engineering*. International Computer Science Series. Pearson.
- Williams, A. (2012). *C++ Concurrency in action - Practical multithreading*. Manning Publications Co.